

FROM KERNEL TO CREATION OS

The operating environment for interactive worlds.

A sequenced path from XACE's deterministic gameplay kernel to a creation, debugging, shipping and ecosystem platform.

THE CENTRAL THESIS

Let AI provide the speed. Put structure, verification and authority underneath it.



2026 EXECUTION / 2027 EXPANSION / 2028+ PLATFORM

The kernel is real. The product is not finished.

The architectural bet is the seed of the future product - but users must experience one coherent creation loop.

PHASE TODAY

XACE CORE

What already exists

- Gameplay represented outside engine-native spaghetti
- Explicit contracts compiled through CGS and SGC
- Authoritative deterministic runtime
- Snapshots, replay, mutation and multiplayer authority
- AI proposes changes; XACE validates and owns execution

PHASE FUTURE

THE COMPLETE SYSTEM

What must emerge

- A human-friendly model of the game
- Instant play, invitation and live iteration
- Causal debugging that feels impossible elsewhere
- Integrated workflows that carry projects through shipping
- Portable mechanics, extensions and network effects

PROMISE DISCIPLINE

CREDIBLE REVOLUTION

One ambitious creator or tiny team can attempt games that previously required much larger technical teams.

Vertical slices in hours or days. Constrained shippable games in weeks or months.

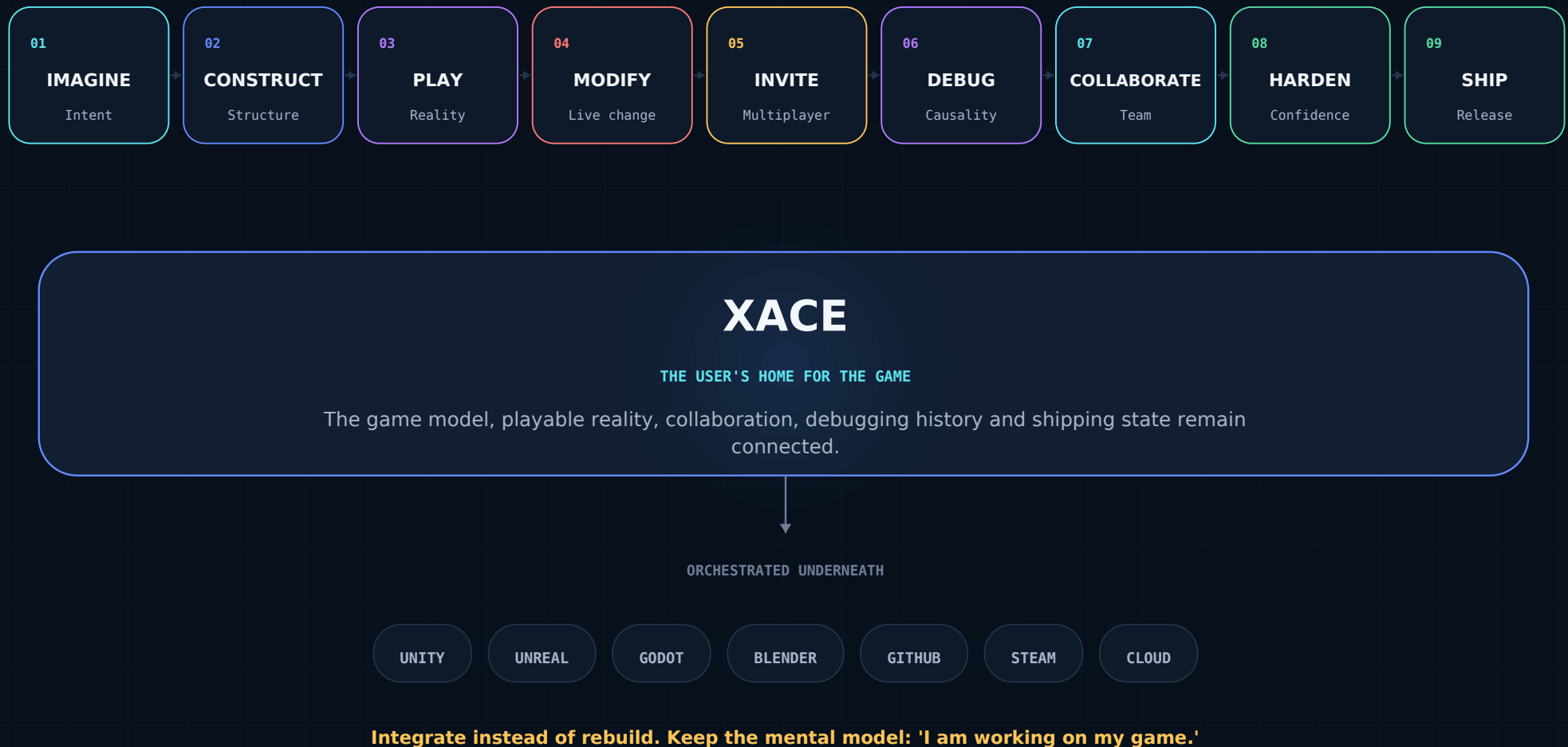
DO NOT PROMISE

A solo novice makes any GTA-scale game in 90 days. AI compresses complexity; it does not erase content, taste, QA or rights.

XACE democratizes production - not copyright ownership.

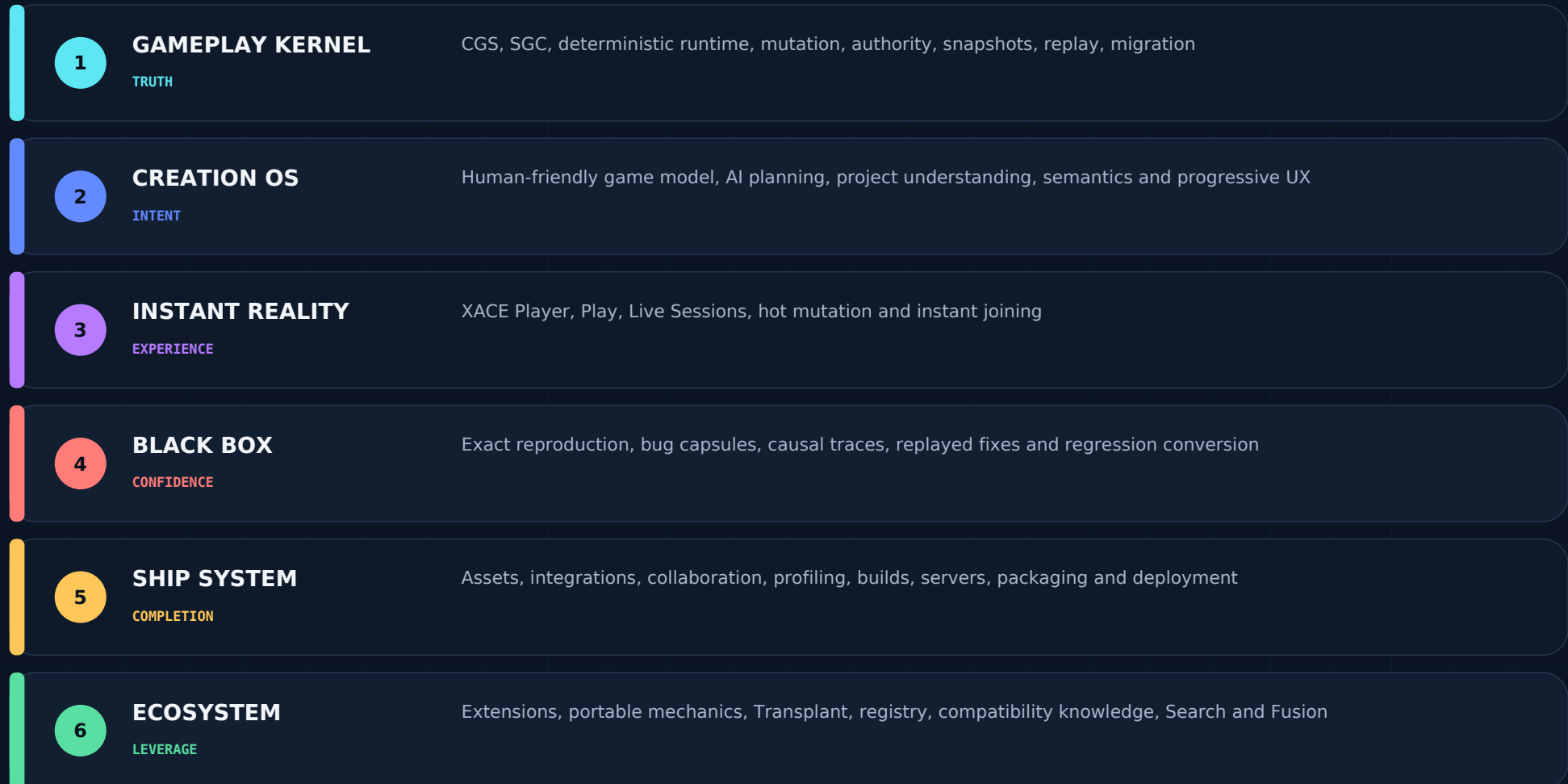
One conceptual loop. Almost no context switching.

XACE should orchestrate the tools underneath while the user stays focused on the game.



The six layers XACE must own

Dominance requires these layers to feel like one system - not six tabs built by six startups.

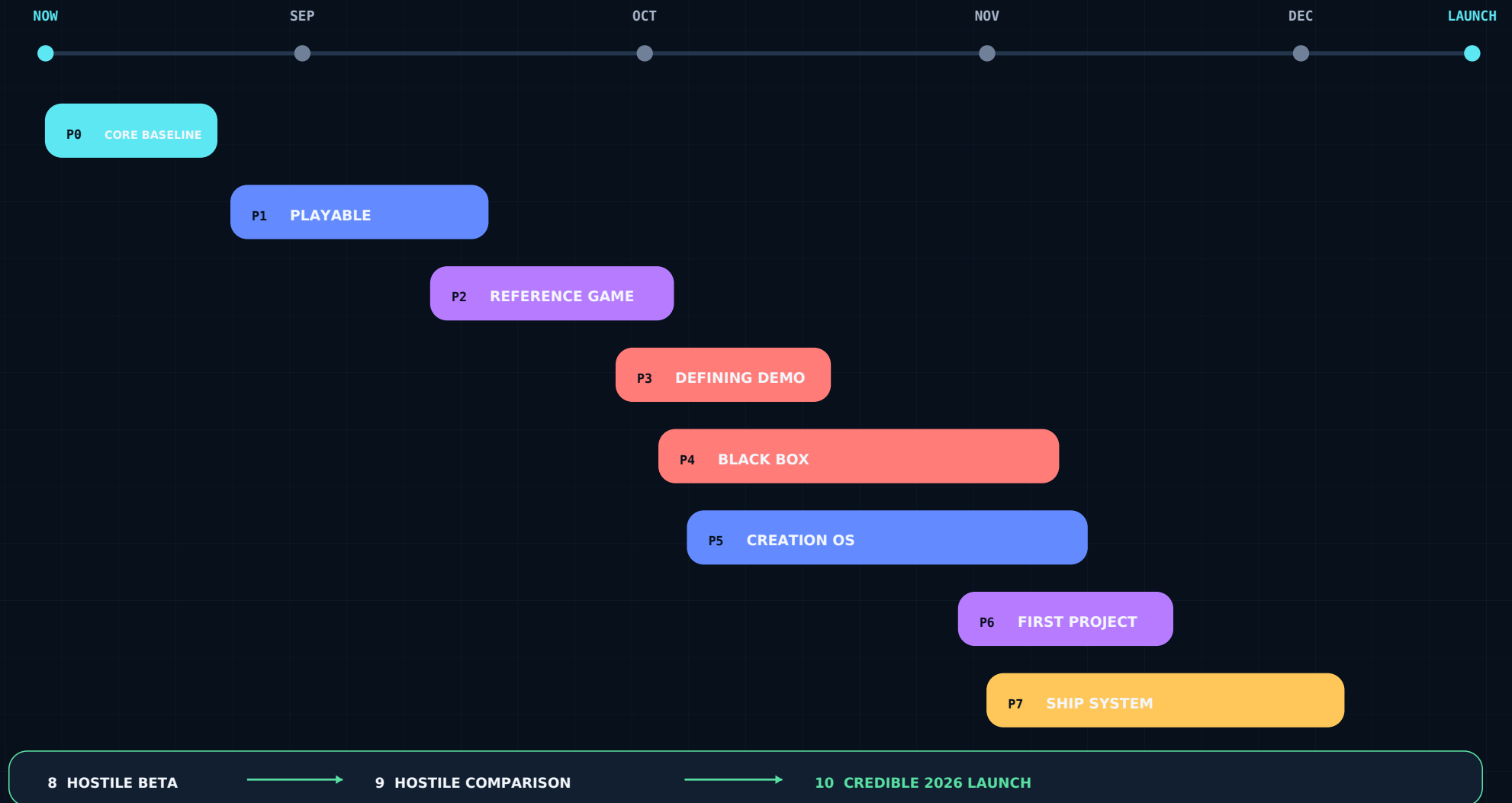


BUILD ORDER

Kernel -> playable reality -> debugging -> creation workflow -> shipping -> ecosystem

The 2026 execution spine

A sequenced program from core baseline to credible public launch.



Freeze the core. Then make XACE playable.

The first transition is from infrastructure to a place where people build and experience games.

PHASE 0

IMMEDIATE / 7-10 DAYS

Finish X10

Complete the 100 tasks, declare a stable baseline, and stop treating the architectural foundation as casual playground code.

DELIVERABLE

XACE Core Baseline v1

- Retained tests and proofs
- Supported boundaries and known limitations
- Versioned artifacts and migration expectations
- A change policy for CGS, SGC and runtime

OPERATING RULE

Change the kernel when justified - not when inspired at 2:47 AM.

PHASE 1

SEPTEMBER 2026

Make XACE playable

Build the minimum first-party experience that makes play and multiplayer the natural continuation of creation.

XACE PLAYER

Render and host enough for immediate iteration.

MINIMAL WORLD CONTRACT

Surfaces, triggers, geometry, volumes, spawns and semantic capabilities.

LIVE SESSIONS

Invite a friend directly from Play.

NORTH STAR

OPEN XACE -> PLAY -> INVITE FRIEND

Build the game. Then build the demo that defines XACE.

The reference game creates scars; the flagship demo turns those scars into a visible product thesis.

PHASE 2

LATE SEP / EARLY OCT

One genuinely good reference game

A small, real multiplayer traversal and combat game - not another synthetic fixture.

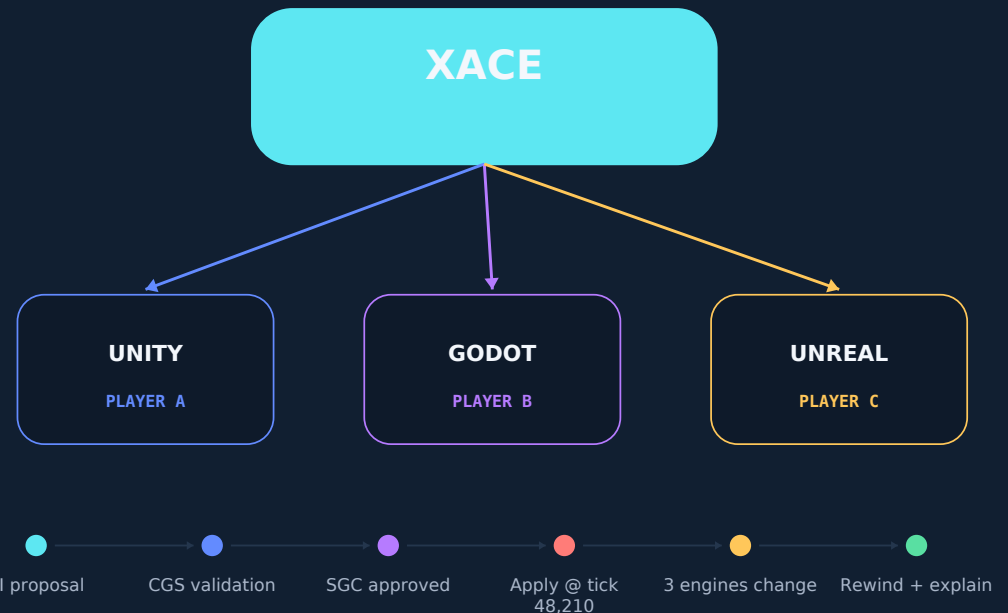
- Wall-running, jumping, weapons and abilities
- Health, respawn, VFX and world interaction
- Packet loss, rollbacks and reconnects
- Simultaneous abilities and weird state transitions
- Live mutations and migration under stress

XACE needs scars before it needs applause.

PHASE 3

OCTOBER 2026

The defining demo



ONE SESSION. ONE MUTATION. THREE ENGINES. EXACT REPLAY.

Make debugging XACE's superpower

A developer who ignores AI hype should still want XACE because debugging without it feels primitive.

BLACK BOX PROMISE

If it happened once, XACE can reproduce it.

1 Incident capsule

- Session + build
- CGS + plan
- Snapshot + inputs
- RNG state
- Network conditions
- Mutations
- State history

2 Causal trace



3 Verified repair

Generate a candidate fix, replay the exact incident, and prove the repair before merging.

- ✓ Bug absent
- ✓ Deterministic
- ✓ Clients agree
- ✓ Regression saved

INCIDENT -> TEST

October - November 2026

Expose the game, not the compiler

The non-technical and professional experiences should be one abstraction with progressive depth.

PHASE 5

OCTOBER - NOVEMBER

MY GAME

● Movement

- Jump
- Sprint
- Wall Run
- Grapple

● Combat

- Health
- Rifle
- Damage
- Respawn

● World

- Main Arena
- Runnable Surfaces
- Spawn Zones

● Multiplayer

- Session
- Teams
- Authority

WALL RUN

Modify Play History

Dependencies Debug

PRO DEPTH

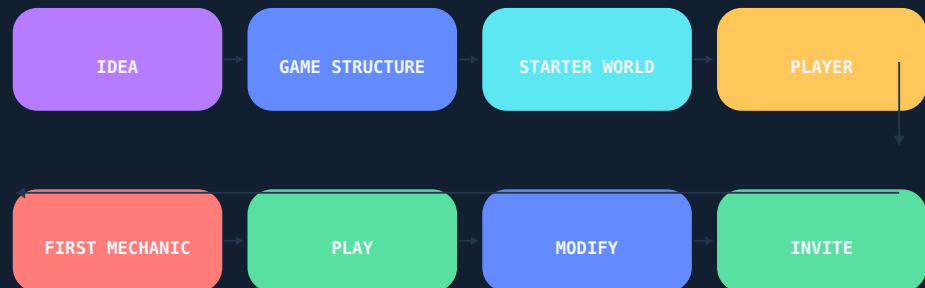
Components, systems, events, reads/writes, network, SGC and state.

PHASE 6

NOVEMBER 2026

Eliminate blank project terror

Do not open with an empty text box. Convert intent into an opinionated, human-approved first playable project.



HUMAN AUTHORITY

AI helps convert intent into structure. The creator approves the game design; AI does not dictate taste.

Win the boring work that decides whether games ship

Future-facing tools die when the user asks: 'Okay, but how do I actually release this?'



Make reality hostile before the market does

The milestone is no longer a passing internal task. It is a user choosing XACE again.

PHASE 8

BEFORE PUBLIC LAUNCH

External hostile beta

10-20 real developers and designers

Give them XACE. Do not hover. Watch where the system helps, blocks, confuses or forces engine-native escape hatches.

- T1** Time to first play
- T2** Time to first meaningful mechanic
- T3** Time to multiplayer
- T4** Time to diagnose a bug
- B** Block rate + escape hatches
- R** Return the next day

PHASE 9

THE HOSTILE COMPARISON

Test the thesis

CONTROL

Developer + Unity +
frontier AI

XACE ARM

Same setup + XACE

SAME UGLY NETWORKED TASK

THESIS

STRUCTURE + AI > UNRESTRICTED AI DEVELOPMENT

DECISION RULE

If normal AI wins badly, do not explain it away. Find out why.

Lead with the architectural difference

Do not compete as another chatbot in an editor. Show speed with engineering-grade control.

PRIMARY LAUNCH MESSAGE

Build games at AI speed.
Keep engineering-grade control.

SHOW

- Describe it, then play it
- Change rules live at an exact tick
- Run one authoritative session across engines
- Rewind, explain and verify every failure
- Retain control as the project grows

DO NOT CLAIM

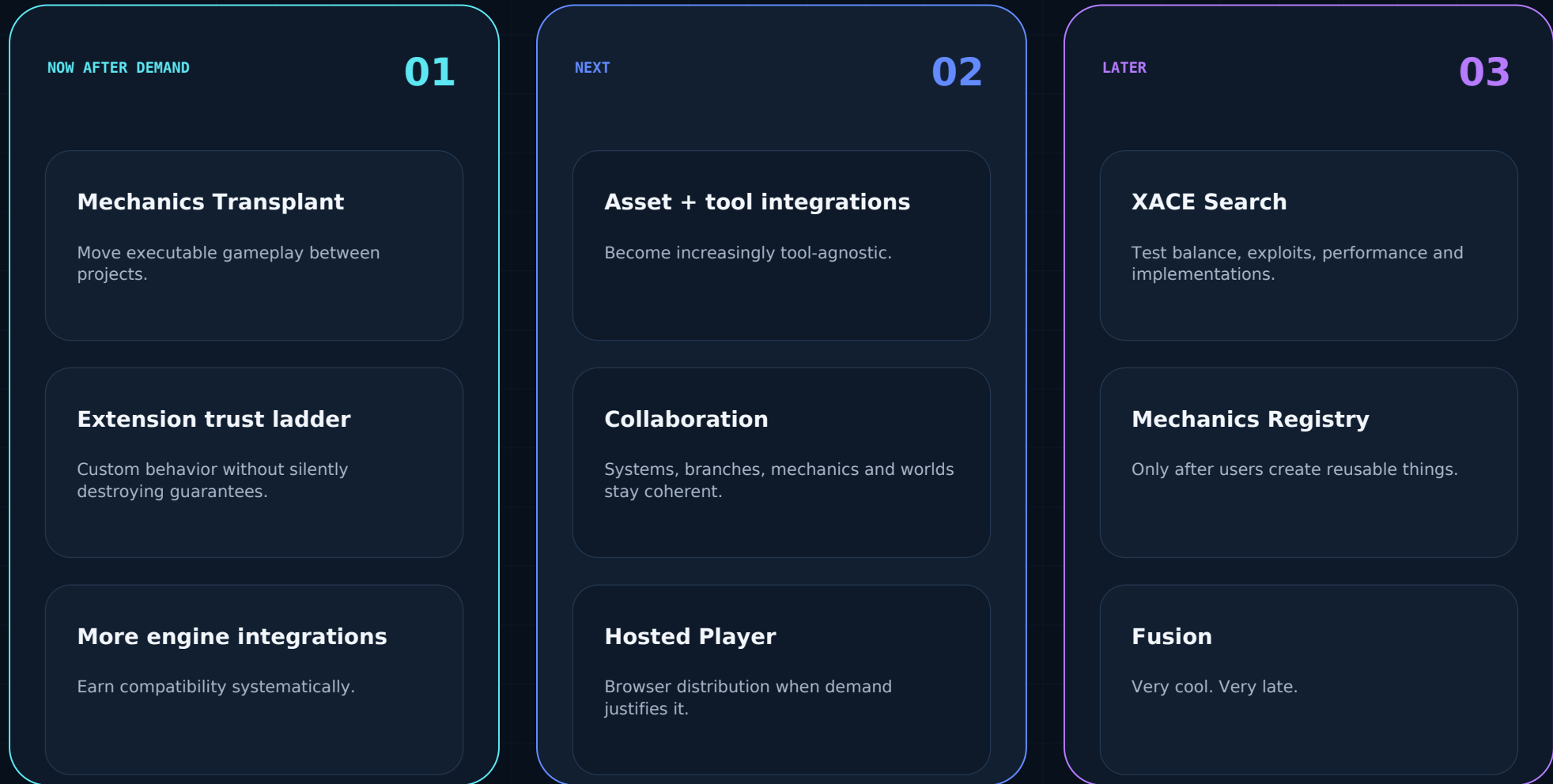
- Every person can build every AAA game instantly
- AI makes content, taste and QA disappear
- XACE replaces every engine and creative tool
- A long feature list is the differentiation

THE DEMO IS THE ARGUMENT.

Launch only after hostile beta and comparison produce a credible proof package.

From a great product to the default choice

If launch proves demand, attack breadth in an order that compounds trust and compatibility.



PRIORITY LOGIC

Portable value -> trusted extensibility -> breadth -> reuse -> network effects

When gameplay becomes portable software

Defensibility emerges from accumulated compatibility, shipped failures, trust and workflow - not from a compiler alone.

A MECHANIC PACKAGE

More than code

SEMANTICS

DEPENDENCIES

SYSTEMS

STATE

EVENTS

NETWORK

WORLD REQS

ASSET BINDINGS

TESTS

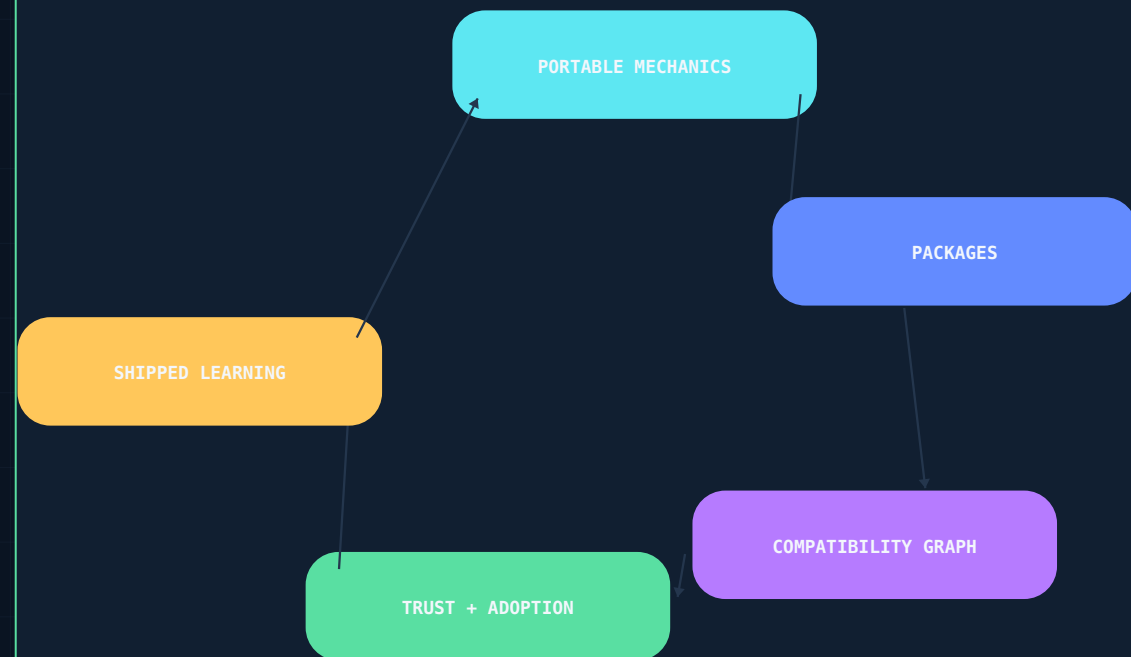
COMPATIBILITY

MIGRATION

PROOF

A reusable mechanic carries everything needed to understand, validate, migrate and run it.

THE DEFENSIBILITY FLYWHEEL

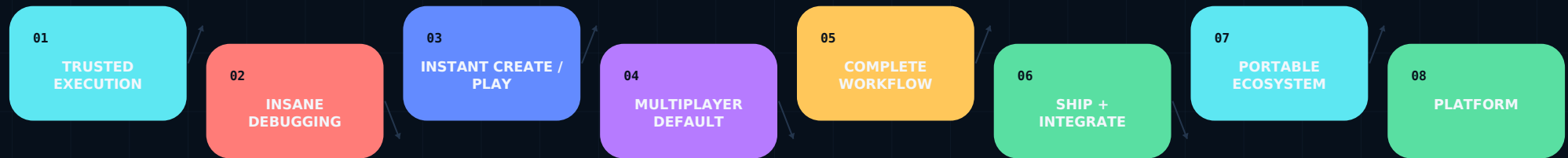


SIMULATIONS

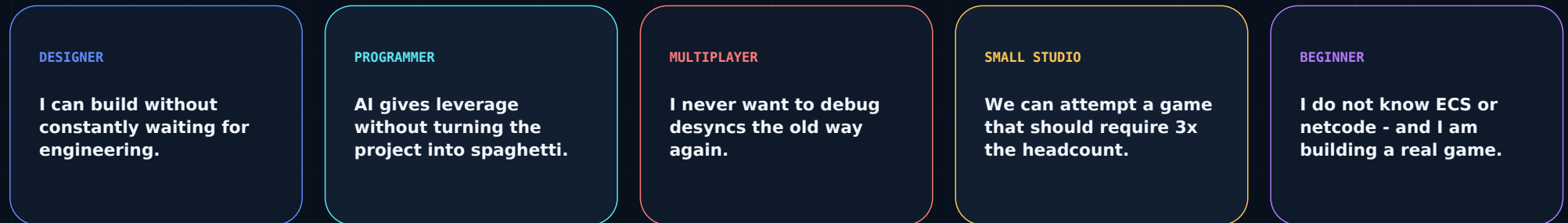
Expand only after winning games: create a domain-specific XACE Simulation Profile.

Become indispensable, then widen the home

The finished vision is not the next build. It is the result of earning the right to add each adjacent layer.



WHAT SUCCESS FEELS LIKE



THE DESTINATION

The kernel is built. Now make it feel like magic - and scale like infrastructure.